

Software Testing

What is Software Testing?

Software Testing means checking if your software:

- Works as expected
- Has no bugs
- Meets user requirements
- Is reliable and safe to use

It's like **inspecting a car** before selling it .You check brakes, engine, lights, and make sure everything works!

What are “Testing Levels”?

When build software, don't just test the whole thing at once.

Test it step by step :from **small pieces** → **combined parts** → **full system** → **final user check**.

These steps are called **Levels of Testing**.

There are **4 main levels**:

1. **Unit Testing**
2. **Integration Testing**
3. **System Testing**
4. **Acceptance Testing**

1.Unit Testing

What is it?

- Testing **the smallest parts** of your code (functions, methods, classes) one by one.
- You test each piece in isolation.

Goal:

Find mistakes in **tiny units** before they combine with others.

Example (FitTrack):

- Test calculateBMI(): Does it always give the right BMI?
- Test addSteps(): Does it correctly update the step count?

Who does it?

Usually developers write unit tests while coding.

Tools:

- JUnit (Java)
- NUnit (.NET)
- PyTest (Python)

2. Integration Testing

What is it?

- After unit testing, you **connect units together** and test if they work as a team.
- Checks the **interfaces** between modules.

Goal:

Catch bugs that happen when parts **talk to each other**.

Example (FitTrack):

- Login module talks to Database. Does logging in save data correctly?
- StepCounter sends data to ProgressBar. Does the bar update correctly?

Common Approaches:

- **Big Bang:** Combine everything, test all at once (risk: harder to find bugs).
- **Incremental:** Add parts step by step (safer).

3. System Testing

What is it?

- Test the **entire software system** as **one piece**, in a real-like environment.
- Check **end-to-end behavior**.

Goal:

Verify that the whole system works according to requirements.

Example (FitTrack):

- Install the app on a phone.
- Create an account, log in, count steps, track water, check daily reports.
- Try normal & edge cases.

Done by:

Dedicated QA testers (not usually the developers).

4. Acceptance Testing

What is it?

- Final check before delivering to the client.
- **Does it meet the user's needs?**

Goal:

Get the client's or end-user's final approval.

Example (FitTrack):

- Client tries all features on their own devices.
- They check: Are the goals clear? Are notifications working? Is the app user-friendly?

Who does it?

- End-users
- Clients
- Sometimes the QA team helps run it too.

Types of Acceptance Testing:

a) Alpha Testing:

- **What it is:** Done by **internal team** in the company.
- **Example:** Developer team tests the app in their own office.
- **Goal:** Catch bugs before showing it to real users.

b) Beta Testing

- **What it is:** Real users test it before full release.
- **Example:** You share a beta version of FitTrack with 20 people.
- **Goal:** Get real feedback, catch hidden bugs.

Other Types of testing:

✓ Smoke Testing

- **What it is:** Basic check "Does it even start?"
- **Example:** Does the app install? Does the home screen open?
- **Goal:** Make sure major parts work before deeper testing.

✓ **Sanity Testing**

- **What it is:** Quick test after a small fix.
- **Example:** You fix login bug → sanity test: does login still work?
- **Goal:** Make sure recent changes didn't break anything.

✓ **Regression Testing**

- **What it is:** Re-test old features after adding new ones.
- **Example:** Add a new workout feature → re-check step counter & water reminder still work.
- **Goal:** Make sure nothing old breaks when you add new.

✓ **Load Testing**

- **What it is:** Test how software handles **lots of users/data**.
- **Example:** 1,000 people use FitTrack at once → does it slow down?
- **Goal:** Find performance limits.

✓ **Stress Testing**

- **What it is:** Push the system **beyond normal limits**.
- **Example:** What happens if 1 million people log in at once?
- **Goal:** See where the system crashes.

✓ **Performance Testing**

- **What it is:** Check speed & response time.
- **Example:** Does the app load each screen in under 2 seconds?
- **Goal:** Make sure it's fast & smooth.

✓ **Usability Testing**

- **What it is:** Test how easy it is for real users to use.
- **Example:** Is the "Start Workout" button easy to find?
- **Goal:** Check user-friendliness.

✓ **Security Testing**

- **What it is:** Find holes hackers could use.
- **Example:** Can someone steal password data?
- **Goal:** Keep user data safe.

✓ **Compatibility Testing**

- **What it is:** Check software works on different devices.

- **Example:** Does FitTrack run well on Android 10, 11, 12?
- **Goal:** Works everywhere!
- **Automation Testing**
 - **What it is:** Use tools to run tests automatically.
 - **Example:** Selenium tests website login daily.
 - **Goal:** Save time, run repeated tests fast.

Summary

Type	What it checks	Example
Unit	Smallest part	One function
Integration	Parts together	Login + DB
System	Whole app	FitTrack full test
Acceptance	Final user check	Client test
Alpha	In-house test	Team test
Beta	Real users test	Pre-release
Smoke	Basic working?	Install, open
Sanity	Quick check	Bug fix test
Regression	Old features OK?	Add new → recheck old
Load	Heavy use	1000 users
Stress	Extreme use	Crash point
Performance	Speed	Response time
Usability	Easy to use?	User-friendly?
Security	Safe?	Hack proof?
Compatibility	Works everywhere?	All devices
White-box	See code	Internal paths
Black-box	Input/output only	Result check
Automation	Tools do tests	Auto login test

Testing Techniques:

There are **two main approaches**:

1. **White-box**
2. **Black-box.**

1.White-Box Testing

What is White-Box Testing?

- White-Box Testing is when the **tester knows exactly how the internal code works** the logic, conditions, loops, structure **everything is visible like an open book** (that's why it's called "**white box**" or sometimes "**glass box**" or "**clear box**" testing).

Key idea:

You test **HOW** the software works **inside**, not just what it does.

What do testers do in White-Box Testing?

Testers look at:

- **Paths in the code** (all possible flows)
- **Conditions** (if-else, switch)
- **Loops** (for, while)
- **Data flow** (how variables change)

They make sure **every line** and **decision** works correctly.

Common White-Box Testing techniques

- **Statement Coverage** — Did every statement run at least once?
- **Branch Coverage** — Did every possible branch (if-else) run?
- **Path Coverage** — Did every path through the code run?
- **Condition Coverage** — Did every condition return both true and false?

Example (FitTrack)

Suppose you have a function:

```
if (steps > goal) {  
    showCongratsMessage();  
} else {  
    showKeepGoingMessage();  
}
```

In **White-Box Testing**, you would check:

- What happens if steps = 5000 and goal = 4000? (should show Congrats)
- What happens if steps = 3000 and goal = 5000? (should show Keep Going)

So you **see and test both paths**.

Tools used

- Unit Testing Frameworks: JUnit, NUnit
- Code Coverage Tools: JaCoCo, Cobertura

Advantages

- Bugs found early (you test logic directly)
- Measures exactly which parts of code were tested (coverage)
- Optimizes code by finding dead or unreachable code
- Great for critical parts like security logic

Limitations

- Not practical for big systems alone (too much code!)
- Can't find missing features — only checks what's there
- Tester must know programming

2. Black-Box Testing

What is Black-Box Testing?

Black-Box Testing is when the **tester does NOT see the code at all**. They only check:

- Inputs
- Outputs
- Functionality

The inside logic is **hidden** — **like a sealed box** — so they test **WHAT it does**, not **HOW it does it**.

Key idea:

- Check if the software **behaves correctly for all possible inputs**, just like a user would.

Common Black-Box Testing Techniques

1. **Equivalence Partitioning** — Split input into valid/invalid groups.

Example: For an age field, valid = 1–99, invalid = -5 or 500.

2. **Boundary Value Analysis** — Test edges: min, max, just inside/outside.

Example: For age 1–99 → test 0, 1, 99, 100.

3. **Decision Table Testing** — If there are rules with different conditions.

Example: Discount rules based on age & membership.

4. **State Transition Testing** — If the system changes states.

Example: Login → logout → forgot password flow.

Example (FitTrack)

Do not see the code.

Test:

- If you enter your weight & height → does BMI come out correct?
- If you enter wrong data → does it show an error?
- If you walk 5000 steps → does it update progress bar?

You test **what user sees**, not how the app calculates inside.

Tools used

- Selenium (web)
- Appium (mobile)
- Manual test cases
- Postman (for API testing)

Advantages

- No need for coding knowledge — even end-users can test
- Good for big systems with complex UI
- Finds missing functions (something forgot to add!)
- Tests actual user experience

Limitations

- Can't check internal code flaws
- Some paths may never be tested if input/output look fine
- Limited coverage if tests are poorly designed

White-Box vs Black-Box

Aspect	White-Box Testing	Black-Box Testing
Code visibility	Code is visible	Code is hidden
Focus on	How it works	What it does
Tester needs to know	Programming	Just requirements
Techniques	Statement, branch, path coverage	Equivalence, boundary, state
Good for	Unit tests, logic errors	System, acceptance tests
Can find	Hidden logic errors	Missing functions, bad behavior
Example	Test both paths of if-else	Enter valid/invalid data in form

When to use each?

White-Box: When you want to test **logic correctness**, especially in **unit testing**.

Black-Box: When you test from the **user's view** — like functional, system, or acceptance testing.

Best practice: Use **BOTH together!**

That's called **Gray-Box Testing** when you know some internals, but test like a user too.

Key Takeaway

- ✓ **White-Box** = Test the *guts*
- ✓ **Black-Box** = Test the *face*
- ✓ **Together** = Total confidence!

Test Case Design

What it means:

A **test case** is a **set of steps** that tells:

- What to test
- What input to use
- What output to expect

Good test cases:

- Cover valid and invalid inputs
- Cover edge cases (extreme values)
- Are repeatable

Example:

Test Case	Input	Expected Output
Test BMI	Weight: 70kg, Height: 1.75m	BMI: 22.9
Invalid BMI	Weight: -5kg, Height: 1.75m	Error message

Debugging vs Testing

Aspect	Testing	Debugging
What	Finding bugs	Fixing bugs
When	During testing phases	After bug is found
Example	Tester runs test case, finds wrong BMI	Developer checks the calculateBMI() code, finds division mistake, fixes it

What is Automation Testing?

- **Automation Testing** means **using software tools/scripts** to run tests on your app **automatically**, instead of manually clicking and typing everything yourself.

In simple words:

- **Manual Testing:** Tester checks everything by hand.
- **Automation Testing:** Computer does the repetitive checks for quickly and repeatedly.

Why do we use Automation Testing?

- To **save time**: machines can run tests faster than humans.
- To **repeat tests easily**: run the same test thousands of times.
- To **catch bugs early**: test every new build automatically.
- To run tests **overnight** or while you do other work.
- To test on **many devices, browsers, or OS versions**.

Where is Automation Useful?

- **Regression Testing:** When you add new features, you check old ones still work.
- **Smoke Testing:** Basic checks to see if a new build is stable.
- **Performance Testing:** Simulate 1,000+ users at once.
- **Data-Driven Testing:** Run the same test with lots of input data.

Example (FitTrack)

Let's say FitTrack app has:

- Login screen
 - Step counter
 - Daily goal tracker
- ✓ Instead of a human logging in 500 times, an **automation tool**:
1. Opens the app,
 2. Enters the username/password,
 3. Clicks login,
 4. Checks if dashboard appears,
 5. Repeats for different accounts.

Popular Automation Tools

Tool	Used For
Selenium	Web apps
Appium	Mobile apps (Android & iOS)
JUnit, TestNG	Unit tests for Java
Cypress	Modern web UI tests
Postman/Newman	API tests

How does it work?

1. **Write Test Scripts:** Write code that tells the tool *what to do* (e.g., click a button, enter text).
2. **Run Tests:** The tool does what the script says — exactly, every time.
3. **Check Results:** Tool compares actual result vs expected result — pass or fail!
4. **Reports:** Automation tools create neat reports showing which tests passed/failed.

Example

- **Manual:**
Tester opens FitTrack → types “Hira” → types password → clicks login → checks dashboard → repeats for 20 users.
- **Automation:**
One script does this for **all 20 users**, clicks, checks, logs results — **in minutes**.

Benefits

- Saves huge time
- Very accurate no human mistakes
- Good for repeated tasks
- Covers large test data fast

Limitations

- Not good for **new UI features** that change daily (scripts break easily).
- Writing automation scripts takes time at first.
- Not every test can be automated (e.g., “Is the UI color nice?” a human must decide).